

AWS CLOUD COMPUTING — FINAL PROJECT

DevOps Pipeline with Container Orchestration

Dockerized Node.js Application on Amazon ECS Fargate

Prepared by

Nigatie Sahlie & Solomon Tibebu

Instructor

Mr. Anteneh Alemu

AWS Region: us-east-1 | July 2026

What We'll Cover



01

Project Overview

Goals, scope, and the technology stack we used

02

System Architecture

How CodePipeline, ECS Fargate, and the ALB fit together

03

Implementation Highlights

Key steps: container → registry → compute → automation

04

Monitoring & Observability

CloudWatch dashboards, alarms, and X-Ray tracing

05

Challenges & Solutions

Real issues we hit and how we resolved them

06

Live Results & Lessons Learned

The working app, cost, and what we'd do next

Project Overview

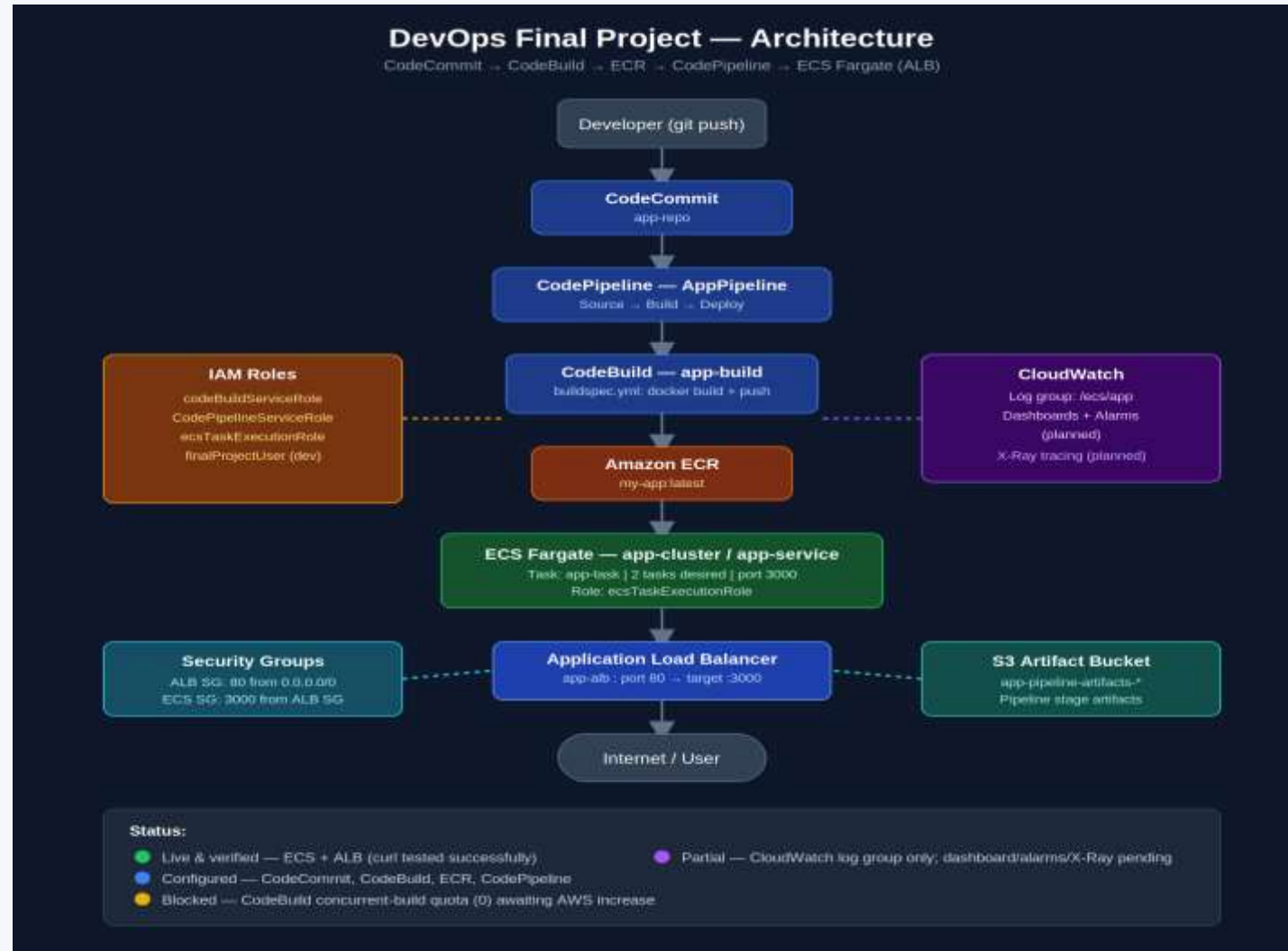
*Option 3 — DevOps Pipeline
with Container Orchestration*

Core Objectives

- 1 Containerize a Node.js/Express app with Docker and a working health check
- 2 Store versioned images in Amazon ECR
- 3 Run the app serverlessly on ECS Fargate — no EC2 management
- 4 Expose it securely via an Application Load Balancer
- 5 Automate build & deploy: CodeCommit → CodeBuild → CodePipeline
- 6 Add observability with CloudWatch and AWS X-Ray

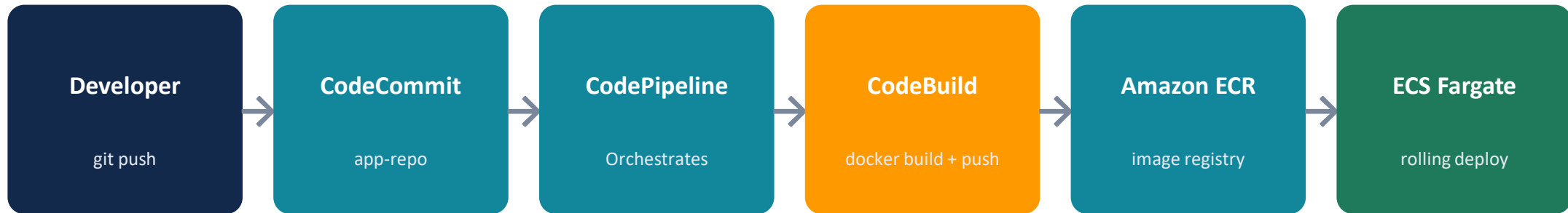
System Architecture

CodeCommit → CodePipeline → CodeBuild → ECR → ECS Fargate → ALB



CI/CD Pipeline Flow

Every git push triggers an automated build and a zero-downtime rolling deployment



Result: 2 Fargate tasks running behind the ALB, updated automatically with zero downtime on every merge to main.

buildspec.yml (excerpt)

```
docker build -t $REPO_URI:latest -f docker/Dockerfile .
docker push $REPO_URI:latest
printf '[{"name":"app","imageUri":"%s"}]' $REPO_URI:latest > imagedefinitions.json
```

Implementation Highlights

Containerization

node:18-alpine base image with a Docker HEALTHCHECK hitting /health every 30s.

IAM Roles

Least-privilege roles for ECS task execution, CodeBuild, and CodePipeline — permissions added only as needed.

Networking

ALB in public subnets (port 80 open); ECS tasks reachable only from the ALB security group (port 3000).

ECS Service

app-cluster running app-task on Fargate — 256 CPU / 512 MB, 2 desired tasks, logs to CloudWatch.

Load Balancing

app-alb + target group (IP target type) health-checking /health before routing traffic.

Automation

CodePipeline: Source (CodeCommit) → Build (CodeBuild) → Deploy (ECS), triggered on every push.

Monitoring & Observability

CloudWatch

- ✓ Log group /ecs/app — centralized container stdout/stderr
- ✓ App-Dashboard — ECS CPU/memory + ALB latency & error widgets
- ✓ Alarm: ECS-High-CPU — triggers above 70% (2 periods)
- ✓ Alarm: ALB-High-Latency — triggers above 1s (2 periods)

AWS X-Ray

- ✓ aws-xray-sdk Express middleware wraps every request
- ✓ X-Ray daemon sidecar container in the task (UDP :2000)
- ✓ Service map + per-request traces in the X-Ray console

```
--threshold 1 --comparison-operator GreaterThanThreshold
nige:app-repo
$ aws cloudwatch describe-alarms --alarm-names ECS-High-CPU ALB-High-Latency
```

DescribeAlarms	
MetricAlarms	
ActionsEnabled	True
AlarmArn	arn:aws:cloudwatch:us-east-1:425124975916:alarm:ALB-High-Latency
AlarmConfigurationUpdatedTimestamp	2026-07-17T07:47:33.827000+03:00
AlarmName	ALB-High-Latency
ComparisonOperator	GreaterThanThreshold
EvaluationPeriods	2
MetricName	TargetResponseTime
Namespace	AWS/ApplicationELB
Period	300
StateReason	Unchecked: Initial alarm creation
StateTransitionedTimestamp	2026-07-17T07:47:33.827000+03:00
StateUpdatedTimestamp	2026-07-17T07:47:33.827000+03:00
StateValue	INSUFFICIENT_DATA
Statistic	Average
Threshold	1.0

MetricAlarms	
ActionsEnabled	True
AlarmArn	arn:aws:cloudwatch:us-east-1:425124975916:alarm:ECS-High-CPU
AlarmConfigurationUpdatedTimestamp	2026-07-17T07:47:31.358000+03:00
AlarmName	ECS-High-CPU
ComparisonOperator	GreaterThanThreshold
EvaluationPeriods	2
MetricName	CPUUtilization
Namespace	AWS/ECS
Period	300
StateReason	Unchecked: Initial alarm creation
StateTransitionedTimestamp	2026-07-17T07:47:31.358000+03:00
StateUpdatedTimestamp	2026-07-17T07:47:31.358000+03:00
StateValue	INSUFFICIENT_DATA
Statistic	Average
Threshold	70.0

Dimensions	
Name	Value
ServiceName	app-service
ClusterName	app-cluster

```
nige:app-repo
$
```

Challenges & Solutions

Real issues encountered during the build — and how each was resolved

Issue	Root Cause	Resolution
ECS tasks failed to launch	ecsTaskExecutionRole didn't exist	Created role with correct trust policy
ResourceInitializationError	CloudWatch log group /ecs/app missing	Created log group before redeploying
ALB URL timed out	No inbound rule for port 80 on ALB security group	Authorized TCP 80 from 0.0.0.0/0
CodeBuild create-project failed	source.type and artifacts.type mismatched	Set both to CODEPIPELINE
Pipeline Build stage failed	CodeBuild concurrent-build quota was 0	Requested a Service Quotas increase

Live Results

Verified end-to-end via the ALB's public DNS name

```
$ curl http://app-alb-*.us-east-1.elb.amazonaws.com/
```

```
{"message":"Hello from ECS Fargate!",  
  "containerId":"ip-172-31-...",  
  "timestamp":"2026-07-16T18:18:06Z"}
```

2/2

Tasks Running

100%

Targets Healthy

~\$41

Est. / Month



Conclusion & Lessons Learned

- IAM least-privilege works best iteratively — grant only what the current error requires
- ECS deployment events point directly at the fix — missing roles, missing log groups
- CodeBuild + CodePipeline requires matching source/artifact types (CODEPIPELINE)
- New AWS accounts often start with service quotas at 0 — separate from IAM permissions
- Verifying each layer independently made root-causing failures fast, not overwhelming

Thank you — Questions?